PCIe 死机问题排查指南

文件标识：RK-KF-YF-C39

发布版本：V1.5.0

日期：2026-04-14

文件密级：□绝密 □秘密 ■内部资料 □公开

免责声明

本文档按“现状”提供，瑞芯微电子股份有限公司（“本公司”，下同）不对本文档的任何陈述、信息和内容的准确性、可靠性、完整性、适销性、特定目的性和非侵权性提供任何明示或暗示的声明或保证。本文档仅作为使用指导的参考。

由于产品版本升级或其他原因，本文档将可能在未经任何通知的情况下，不定期进行更新或修改。

商标声明

“Rockchip”、“瑞芯微”、“瑞芯”均为本公司的注册商标，归本公司所有。

本文档可能提及的其他所有注册商标或商标，由其各自拥有者所有。

版权所有 © 2026 瑞芯微电子股份有限公司

超越合理使用范畴，非经本公司书面许可，任何单位和个人不得擅自摘抄、复制本文档内容的部分或全部，并不得以任何形式传播。

瑞芯微电子股份有限公司

Rockchip Electronics Co., Ltd.

地址： 福建省福州市铜盘路软件园A区18号

网址： www.rock-chips.com

客户服务电话： +86-4007-700-590

客户服务传真： +86-591-83951833

客户服务邮箱： fae@rock-chips.com

前言

概述

产品版本

芯片名称	内核版本
所有非RK3399的、带PCIe控制器的芯片	All

读者对象

本文档（本指南）主要适用于以下工程师：

技术支持工程师

软件开发工程师

修订记录

版本号	作者	修改日期	修改说明
V1.0.0	林涛	2025-04-01	初始版本
V1.1.0	林涛	2025-04-18	添加复位检查
V1.2.0	林涛	2025-06-01	添加PHY异常说明和DBI写使能的打印
V1.3.0	林涛	2025-08-13	添加硬件检查正常后的IOMUX检查项; 修改IO标签
V1.4.0	林涛	2026-01-27	修改表格格式，添加RK3572超时配置说明
V1.5.0	林涛	2026-04-14	添加NOC超时后排查和处理

目录

PCIe 死机问题排查指南

1. 排查流程
2. 排查项目
   - 2.1 查看log是否有synchronous external abort
   - 2.2 测量PHY供电和100MHz时钟
   - 2.3 查看PCIe LTSSM状态机
   - 2.4 检查pending传输
   - 2.5 添加ordering rule
   - 2.6 时钟/PD常开
   - 2.7 禁用ASPM
   - 2.8 提高vdd_logic电压
   - 2.9 检查softrst配置
   - 2.10 添加FTS补丁
   - 2.11 禁用此设备L0s功能
   - 2.12 检查IOMUX配置
   - 2.13 查看err event
   - 2.14 开启s2r流程打印
   - 2.15 是否访问DBI寄存器死机
   - 2.16 查看phy状态是否被切换
   - 2.17 休眠唤醒分层测试
3. 常见因素说明
   - 3.1 Producer-Consumer模型异常
   - 3.2 L0s的支持异常
   - 3.3 总线阻塞
   - 3.4 链路或者外设异常
   - 3.5 PHY的参考时钟异常
   - 3.6 功能引脚被误激活
4. 解决方法
5. 附录
   - 5.1 开发资源获取地址
   - 5.2 LTSSM状态机
   - 5.3 Debugfs导出信息解析表

1. 排查流程

PCIe 死机问题是指系统使用过程中，因操作 PCIe 设备或怀疑 PCIe 设备而导致的死机。本文档旨在针对公司内部的技术支持人员或者软件工程师：

✅ 构建系统性排查框架

✅ 减少平均故障处理时长（MTTR）

✅ 降低重复性误判率

✅ 明确软硬件责任边界判定标准

总体排查流程如下：

2. 排查项目

请按照`排查流程`中的流向顺序，针对核对列表中所列实验，逐一展开实验并填写结论，以便更好的追踪和梳理实验。

核对列表	核对结论	备注
查看log是否有synchronous external abort		
测了PHY供电和100MHz时钟		
查看PCIe LTSSM状态机		
检查pending传输		
添加ordering rule		
时钟/PD常开		
禁用ASPM		
提高vdd_logic电压		
检查softrst配置		
添加FTS补丁		
禁用此设备L0s功能		
检查IOMUX配置		
查看err event		
开启s2r流程打印		
是否访问DBI寄存器死机		
查看phy状态是否被切换		
休眠唤醒分层测试		

2.1 查看log是否有synchronous external abort

在部分设计有NOC超时的平台，如果EP设备在收取了来自主机的访问后发生异常，无法将数据回传给主机，主机经历较长时间的等待后，将发生synchronous external abort，并且将访问调用栈导出，以供debug之用。

```
[03/26 02:18:21][   11.661045][  T696] [dhd] dhdpcie_readshared: Failed to find
CA7 core!
[03/26 02:18:21][   11.661146][  T696] [dhd] dhdpcie_readshared: address
(0xfa1905e6) of pciedev_shared invalid
[03/26 02:18:21][   11.661159][  T696] [dhd] dhdpcie_readshared: Waited 2000000
usec, dongle is not ready
[03/26 02:18:21][   11.661170][  T696] [dhd] dhd_bus_init :Shared area read
failed
[03/26 02:18:21][   11.661200][  T696] [dhd] dhd_bus_start, dhd_bus_init failed
-1
[03/26 02:18:21][   11.661211][  T696] [dhd] dhdpcie_init: dhd_bus_start()
failed
[03/26 02:18:21][   11.661223][  T696] [dhd] dhdpcie_bus_remove_prep: making
DHD_BUS_DOWN
```

```
[03/26 02:18:22][   12.173819][  T696] Internal error: synchronous external
abort: 0000000096000210 [#1] PREEMPT SMP
[03/26 02:18:22][   12.174619][  T696] Modules linked in: bcmdhd(O+) r8168
[03/26 02:18:22][   12.177069][  T696] CPU: 3 UID: 1010 PID: 696 Comm:
android.hardwar Tainted: G   M       O       6.12.58+ #1
[03/26 02:18:22][   12.177942][  T696] Tainted: [M]=MACHINE_CHECK,
[O]=OOT_MODULE
[03/26 02:18:22][   12.177945][  T696] Hardware name: Rockchip RK3572 EVB1 V10
Board (DT)
[03/26 02:18:22][   12.177949][  T696] pstate: 604000c5 (nZCv daIF +PAN -UAO -
TCO -DIT -SSBS BTYPE=--)
[03/26 02:18:22][   12.179554][  T696] pc : pci_generic_config_read+0x74/0xc0
[03/26 02:18:22][   12.183720][  T696] lr : pci_generic_config_read+0x3c/0xc0
[03/26 02:18:22][   12.184228][  T696] sp : ffffffc085b7b370
[03/26 02:18:22][   12.184589][  T696] x29: ffffffc085b7b370 x28:
ffffffdf8daa8000 x27: ffffffdf8daa8000
[03/26 02:18:22][   12.185287][  T696] x26: ffffff80eb1b9000 x25:
ffffffdf8daa8000 x24: 00000000000000c0
[03/26 02:18:22][   12.185981][  T696] x23: 0000000000000000 x22:
000000000000005a x21: ffffff81fff84000
[03/26 02:18:22][   12.186676][  T696] x20: 0000000000000002 x19:
ffffffc085b7b3e4 x18: ffffffc083c5d130
[03/26 02:18:22][   12.187371][  T696] x17: 00000000a43ffce2 x16:
00000000a43ffce2 x15: 000000000000007e
[03/26 02:18:22][   12.188066][  T696] x14: fffffffffffffffe x13:
fffffffffffffffe x12: ffffff80c22a86d8
[03/26 02:18:22][   12.188762][  T696] x11: 0000000000000104 x10:
0000000000000000 x9 : 0f4a95618da76f00
[03/26 02:18:22][   12.189460][  T696] x8 : 0f4a95618da76f00 x7 :
7f7f7f7f7f7f7f7f x6 : 302e30303a31313a
[03/26 02:18:22][   12.190155][  T696] x5 : 0000000000000000 x4 :
0000000080000000 x3 : 0000000000000004
[03/26 02:18:22][   12.190848][  T696] x2 : 0000000000000000 x1 :
ffffffc080f00000 x0 : ffffffc08110005a
[03/26 02:18:22][   12.191544][  T696] Call trace:
[03/26 02:18:22][   12.191828][  T696]  pci_generic_config_read+0x74/0xc0
[03/26 02:18:22][   12.192301][  T696]  dw_pcie_rd_other_conf+0x2c/0xa4
[03/26 02:18:22][   12.192757][  T696]  pci_bus_read_config_word+0x8c/0xf0
[03/26 02:18:22][   12.193232][  T696]  pci_read_config_word+0x40/0x50
[03/26 02:18:22][   12.193671][  T696]  __pci_write_msi_msg+0x6c/0x1e8
[03/26 02:18:22][   12.194112][  T696]  pci_msi_domain_write_msg+0x2c/0x3c
[03/26 02:18:22][   12.194583][  T696]  msi_domain_deactivate+0x50/0x7c
[03/26 02:18:22][   12.195033][  T696]  __irq_domain_deactivate_irq+0x4c/0x6c
[03/26 02:18:22][   12.195527][  T696]  irq_domain_deactivate_irq+0x24/0x44
[03/26 02:18:22][   12.195999][  T696]  free_irq+0x190/0x310
[03/26 02:18:22][   12.196361][  T696]  dhdpcie_free_irq+0x48/0x148 [bcmdhd]
[03/26 02:18:22][   12.197705][  T696]  dhdpcie_bus_release+0x24c/0x724 [bcmdhd]
[03/26 02:18:22][   12.198773][  T696]  dhdpcie_pci_probe+0x450/0x1204 [bcmdhd]
[03/26 02:18:22][   12.199714][  T696]  pci_device_probe+0xb0/0x13c
[03/26 02:18:22][   12.200138][  T696]  really_probe+0x1b4/0x3a0
[03/26 02:18:22][   12.200536][  T696]  __driver_probe_device+0x98/0xd4
[03/26 02:18:22][   12.200984][  T696]  driver_probe_device+0x3c/0x218
[03/26 02:18:22][   12.201420][  T696]  __driver_attach+0x10c/0x1e8
```

一旦确认发生如上情况，请返回排查流程，由流程2至流程4，逐步排查导致EP设备异常的原因。

2.2 测量PHY供电和100MHz时钟

- PCIe PHY需要两路供电，分别为0V9和1V8，请测量复现问题的过程中是否有塌陷或者异常纹波。
- 3.0的PHY请测量输入的100MHz时钟是否中断过(一般可测量外部晶振的电源使能IO)
- 2.0的PHY请测量异常后，输出100MHz时钟是否还有持续输出

2.3 查看PCIe LTSSM状态机

- 可通过查找 TRM 手册获取要排查的 PCIe 控制器的 APB 寄存器地址，也可以从相应的 dtsi 文件中查找。

  以RK3568 PCIe2x1控制器为例

  ```
  # TRM Address Mapping 章节
  | PCIe2x1_APB | 0xFE260000 | 64KB |

  # arch/arm64/boot/dts/rockchip/rk356x.dtsi
  pcie2x1: pcie@fe260000 {
      ...
  };
  ```

- 读取LTSSM状态数值，地址为APB寄存器地址加上0x300的偏移

  ```
  # U-Boot
  md.l 0xfe260300 1

  # Linux/Android
  io -4 0xfe260300
  ```

- 前一步输出的数值，最后5个bit为LTSSM状态值，请查阅附录中的LTSSM状态机。

2.4 检查pending传输

下表列举了部分芯片的 PCIe 控制器传输阻塞查询寄存器，我们称为pending A寄存器。借助这些寄存器，能够查询当前片内总线上是否存在尚未完成的 PCIe 传输。当寄存器对应的比特位为 0 时，意味着当前该控制器存在未完成的传输。

芯片	控制器	寄存器地址	比特位
RK3562	pcie2x1	io -4 0xff030438	16
RK3576	pcie0	io -4 0x2600a074	16(dbi), 17(slave)
RK3576	pcie1	io -4 0x2600a074	18(dbi), 19(slave)
RK3588	PCIe1L0	io -4 0xfd58c18c	14
RK3588	PCIe1L1	io -4 0xfd58c18c	13
RK3588	PCIe1L2	io -4 0xfd58c18c	12
RK3588	PCIe3x2	io -4 0xfd58c18c	11
RK3588	PCIe3x4	io -4 0xfd58c18c	10

表格未列举的芯片因为没有预留总的标志位，所以需要通过读取PCIe控制器APB地址 + 0x104的寄存器，我们称为pending B寄存器，来进一步明确PCIe控制器层面被阻塞且未完成的传输类型 。仍然以RK3568 PCIe2x1控制器为例(pcie2x1: pcie@fe260000) , 该寄存器为0xfe260104，对应比特若为1则表示当前所示传输类型被阻塞。

比特	描述
7	Receive TLP requests is pending
6	DMA transfer is pending
5	AXI Slave DBI transfer is pending
4	AXI Slave non-DBI transfer is pending

- case 1：如果查看pending A寄存器存在未完成传输，则尝试保留一个CPU用于后续的debug：

```
--- a/arch/arm64/boot/dts/rockchip/rk3588-android.dtsi
+++ b/arch/arm64/boot/dts/rockchip/rk3588-android.dtsi
@@ -6,7 +6,7 @@
  / {
        chosen: chosen {
-               bootargs = "earlycon=uart8250,mmio32,0xfeb50000 console=ttyFIQ0
irqchip.gicv3_pseudo_nmi=0";
+               bootargs = "earlycon=uart8250,mmio32,0xfeb50000 console=ttyFIQ0
irqchip.gicv3_pseudo_nmi=0 isolcpus=3";
        };
```

- case 2：如果查看pending A寄存器没有未完成的PCIe传输，则需要进一步读取pending B寄存器，查看是否控制器层面存在由EP设备发送来的、被阻塞的传输。如果是pending A指示总线不存在未完成传输而pending B指示控制器层面存在未完成传输，大概率是EP的传输模型存在违反ordering rule的情况，优先尝试按照`2.4 添加ordering rule`章节，用io命令(或者jtag相关的配置寄存器命令)临时调整ordering rule寄存器，一般可以使得卡住的CPU得到释放。
- case 3：如果查看pending A寄存器没有未完成的PCIe传输，且pending B寄存器也不存在未完成传输，但是CPU就是显示卡在访问PCIe的BAR空间，则目前已知的情况是可能PCIe控制器在被访问的同时被复位，其将AXI的AR-ready（读地址通道就绪信号）和AW-ready（写地址通道就绪信号）拉低，导致AXI发向PCIe控制器的读写操作命令无法发出就挂死；

- case 4：如果查看pending A寄存器没有未完成的PCIe传输，且pending B寄存器也不存在未完成传输，但是CPU就是显示卡在访问PCIe的DBI空间，则目前已知的情况是可能PCIe的PHY出现异常，推荐优先测量现场PCIe的PHY的100MHz参考时钟是否正常。

2.5 添加ordering rule

```
添加ordering rule限制的方法有两种，可以用io命令或者修改代码方式

######################io命令方式如下######################
1. 查询PCIe控制器，获得dbi寄存器基地址，我们以RK3568 pcie2x1为例
# TRM Address Mapping 章节查询 PCIe2x1的DBI地址，有两组都可以用
| PCIe2x1_DBI | 0x3C0000000 | 4MB |
| PCIe2x1_DBI |  0xF6800000 | 4MB |

# 或者从rk3568.dtsi中获取pcie-dbi所对应的地址是0x3c0000000
pcie2x1: pcie@fe260000 {
    compatible = "rockchip,rk3568-pcie", "snps,dw-pcie";
    ...
    reg = <0x3 0xc0000000 0x0 0x400000>, <0x0 0xfe260000 0x0 0x10000>;
    reg-names = "pcie-dbi", "pcie-apb";
    ...
};

2. 临时调整Ordering rule寄存器，允许CPLs超过Pd，寄存器为dbi地址+0x8b4：
io -4 0x3C00008b4 0xff00  # 0x3c0000000 + 0x8b4

3. Ordering rule寄存器详解：
_____________________________________________________________________________
___
|[0: 7] | Determines if NP can pass halted P queue. (Completion Passing Posted规
则) |
|       | 0: NP can not pass P (recommended)
   |
|       | 1: NP can pass P
   |
|_______|_____________________________________________________________________
___ |
|[8:15] | Determines if CPL can pass halted P queue.(Non-Posted Passing Posted规
则)  |
|       | 0: CPL can not pass P (recommended)
    |
|       | 1: CPL can pass P
    |
|_______|_____________________________________________________________________
___ |

######################修改代码方式如下######################
--- a/drivers/pci/controller/dwc/pcie-dw-rockchip.c
+++ b/drivers/pci/controller/dwc/pcie-dw-rockchip.c
@@ -226,6 +226,7 @@ static inline void rk_pcie_set_rc_mode(struct rk_pcie
*rk_pcie)
                disable_aspm_l1ss(rk_pcie);
        }
        rk_pcie_writel_apb(rk_pcie, 0x0, 0xf00040);
+       dw_pcie_writel_dbi(rk_pcie->pci, PCIE_PL_ORDER_RULE_CTRL_OFF, 0xff00);
```

```
}
```

2.6 时钟/PD常开

时钟和PD常开可以有三种修改方式，第一种是dts中添加属性，第二种是命令行配置，第三种是直接修改代码。我们建议是使用直接修改代码方式，这样更保险。

仍然以RK3568为例：

```
# 5.10内核 Linux平台
--- a/arch/arm64/boot/dts/rockchip/rk3568-linux.dtsi
+++ b/arch/arm64/boot/dts/rockchip/rk3568-linux.dtsi
@@ -13,7 +13,7 @@
        };

chosen: chosen {
-       bootargs = "earlycon=uart8250,mmio32,0xfe660000 console=ttyFIQ0
root=PARTUUID=614e0000-0000 rw rootwait";
+       bootargs = "earlycon=uart8250,mmio32,0xfe660000 console=ttyFIQ0
root=PARTUUID=614e0000-0000 rw rootwait clk_gate.always_on=1
pm_domains.always_on=1";
        };

# 5.10内核 Android平台
--- a/arch/arm64/boot/dts/rockchip/rk3568-android.dtsi
+++ b/arch/arm64/boot/dts/rockchip/rk3568-android.dtsi
@@ -6,7 +6,7 @@

 / {
        chosen: chosen {
-               bootargs = "earlycon=uart8250,mmio32,0xfe660000
console=ttyFIQ0";
+               bootargs = "earlycon=uart8250,mmio32,0xfe660000 console=ttyFIQ0
clk_gate.always_on=1 pm_domains.always_on=1";
        };

# 命令行方式开启(Y)和关闭(N),回读确认
echo Y > sys/module/clk_gate/parameters/always_on
echo Y > sys/module/pm_domains/parameters/always_on
cat sys/module/clk_gate/parameters/always_on
cat sys/module/pm_domains/parameters/always_on

echo N > sys/module/clk_gate/parameters/always_on
echo N > sys/module/pm_domains/parameters/always_on
cat sys/module/clk_gate/parameters/always_on
cat sys/module/pm_domains/parameters/always_on

#其他内核版本，建议直接修改代码
--- a/drivers/clk/clk-gate.c
+++ b/drivers/clk/clk-gate.c
@@ -64,6 +64,9 @@ static void clk_gate_endisable(struct clk_hw *hw, int enable)
        unsigned long flags;
        u32 reg;

+       if (enable == 0)
```

```
+               return 0;
+

--- a/drivers/soc/rockchip/pm_domains.c
+++ b/drivers/soc/rockchip/pm_domains.c
@@ -981,6 +981,9 @@ static int rockchip_pd_power(struct rockchip_pm_domain *pd,
bool power_on)
        int ret = 0;
        struct generic_pm_domain *genpd = &pd->genpd;

+       if (power_on == 0)
+               return 0;
+
```

如果本实验有效

- 继续进行二分法，保留时钟常开或者PD常开，看具体是哪一个影响
- 如果定位是时钟常开有效，则去掉PD的补丁，把被关闭的时钟打印出来，在卡死前后查看目标对象。如果定位是PD常开有效，亦如同操作。

```
#打印被关闭时钟项
--- a/drivers/clk/clk-gate.c
+++ b/drivers/clk/clk-gate.c
@@ -64,6 +64,9 @@ static void clk_gate_endisable(struct clk_hw *hw, int enable)
        unsigned long flags;
        u32 reg;

+       if (enable == 0)
+               pr_err("clk disable %s\n", clk_hw_get_name(hw));
+

#打印被关闭PD项
--- a/drivers/soc/rockchip/pm_domains.c
+++ b/drivers/soc/rockchip/pm_domains.c
@@ -1012,6 +1012,9 @@ static int rockchip_pd_power(struct rockchip_pm_domain
*pd, bool power_on)
        int ret = 0;
        struct generic_pm_domain *genpd = &pd->genpd;

+       if (power_on == 0)
+               pr_err("PD disable: %s\n", genpd->name);
+
        if (pm_domain_always_on && !power_on)
                return 0;
```

如果定位到死机前后有若干个时钟被关闭导致，则去掉之前的补丁，使用以下两种方式之一进行详细定位：

- 使用二分法将定位到的可疑目标时钟进行引用计数递增，继续拷机，直到找到问题时钟和问题模块。

```
# 例如异常之前有尝试关闭aclk_jdec，使用如下操作可以单独保持其时钟，其他时钟节点亦如是操作
echo 1 > /sys/kernel/debug/clk/aclk_jdec/clk_enable_count
或者
echo 1 > /sys/kernel/debug/clk/aclk_jdec/clk_prepare_enable;
```

- 使用jtag连接，然后根据TRM，找到目标时钟的寄存器，逐个打开，再检查pending传输，看PCIe的pending位是否消失，若消失则表明找到目标时钟。

```
# openOCD集成了noc命令可以查看pending传输
> noc
0xfd5c8180: 000003ff f00afff7 e477e0f0 0ffb1c
0xfd5c8180: 000003ff f01bbff e477e0f0 0ffffe1d is normal
jpegdec pending
gic600_axim0 pending
vop_m3 pending
usb3_2 pending
pcie3_110 pending
mmuphp4 pending

# 打开aclk_jdec
> io -4 -l 0x4 0xfd7c08c2 0xffff0000

# 再查看是否pending消失，若消失则aclk_jdec是目标时钟
> noc
```

2.7 禁用ASPM

```
# 方式一：去除以下ASPM的config配置
CONFIG_PCIEASPM=n
CONFIG_PCIEASPM_POWERSAVE=n

# 方式二：添加pcie_aspm=off
# Android平台
--- a/arch/arm64/boot/dts/rockchip/rk3568-android.dtsi
+++ b/arch/arm64/boot/dts/rockchip/rk3568-android.dtsi
@@ -6,7 +6,7 @@

 / {
        chosen: chosen {
-               bootargs = "earlycon=uart8250,mmio32,0xfe660000
console=ttyFIQ0";
+               bootargs = "earlycon=uart8250,mmio32,0xfe660000 console=ttyFIQ0
pcie_aspm=off";
        };
# Linux平台
--- a/arch/arm64/boot/dts/rockchip/rk3568-linux.dtsi
+++ b/arch/arm64/boot/dts/rockchip/rk3568-linux.dtsi
@@ -13,7 +13,7 @@ aliases {
        };

        chosen: chosen {
-               bootargs = "earlycon=uart8250,mmio32,0xfe660000 console=ttyFIQ0
root=PARTUUID=614e0000-0000 rw rootwait";
```

```
+               bootargs = "earlycon=uart8250,mmio32,0xfe660000 console=ttyFIQ0
root=PARTUUID=614e0000-0000 rw rootwait pcie_aspm=off";
        };
```

禁用ASPM之后，请使用lspci工具，检查RC LnkCtl寄存器中的ASPM功能是否已经关闭

```
# lspci -vvv
0002:20:00.0 Class 0604: Device 1d87:3566 (rev 01)
        ...
        Capabilities: [70] Express (v2) Root Port (Slot-), MSI 08
                DevCap: MaxPayload 256 bytes, PhantFunc 0
                        ExtTag- RBE+
                DevCtl: Report errors: Correctable- Non-Fatal- Fatal-
Unsupported-
                        RlxdOrd+ ExtTag- PhantFunc- AuxPwr- NoSnoop-
                        MaxPayload 128 bytes, MaxReadReq 512 bytes
                DevSta: CorrErr- UncorrErr- FatalErr- UnsuppReq- AuxPwr-
TransPend-
                LnkCap: Port #0, Speed 8GT/s, Width x2, ASPM L1, Exit Latency
L0s <4us, L1 <64us
                        ClockPM- Surprise- LLActRep+ BwNot- ASPMOptComp+
                LnkCtl: ASPM Disabled; RCB 64 bytes Disabled- CommClk-  ====>
ASPM Disabed 确认生效!!!
```

2.8 提高vdd_logic电压

仍然以RK3568为例，将最低电压和初始电压都提高。当时要特别注意，不同的芯片平台对此电压的上限有不同的限制，请咨询硬件后进行配置。

```
--- a/arch/arm64/boot/dts/rockchip/rk3568-evb.dtsi
+++ b/arch/arm64/boot/dts/rockchip/rk3568-evb.dtsi
@@ -1168,9 +1168,9 @@
                        vdd_logic: DCDC_REG1 {
                                regulator-always-on;
                                regulator-boot-on;
-                               regulator-min-microvolt = <500000>;
+                               regulator-min-microvolt = <1000000>;
                                regulator-max-microvolt = <1350000>;
-                               regulator-init-microvolt = <900000>;
+                               regulator-init-microvolt = <1000000>;
                                regulator-ramp-delay = <6001>;
                                regulator-initial-mode = <0x2>;
                                regulator-name = "vdd_logic";
```

2.9 检查softrst配置

查询芯片TRM手册CRU章节，找到对应控制器所在的CRU_SOFTRST_CONXX，逐一核对复位配置，是否存在被复位情况。部分单元被复位，如PIPE，可导致DBI和BAR空间无法访问但APB可访问的现象。

2.10 添加FTS补丁

```
#如果内核有如下补丁：
commit 5ef8309d89376c061650c7363572c4d749fbfe85
Author: Shawn Lin <shawn.lin@rock-chips.com>
Date:   Fri Dec 6 17:47:42 2024 +0800

    PCI: rockchip: dw: Enable L0s capability

    Signed-off-by: Shawn Lin <shawn.lin@rock-chips.com>
    Change-Id: I53f82c357045f883ddb03e5645aca207f7a7fd92

#请确认已经合并了如下修复，针对6.1内核之后版本
commit 074134053ecdf463e10a5ce42cf7104ff66237ba
Author: Shawn Lin <shawn.lin@rock-chips.com>
Date:   Fri Feb 28 10:02:12 2025 +0800

    PCI: rockchip: dw: Increase N_FTS needed by L0s exit

    ....

    Change-Id: I071c27ef6a3ac504b9ed82b748b41ec55bfd5fc5
    Signed-off-by: Shawn Lin <shawn.lin@rock-chips.com>

diff --git a/drivers/pci/controller/dwc/pcie-dw-rockchip.c
b/drivers/pci/controller/dwc/pcie-dw-rockchip.c
index 7329074..f72aab5 100644
--- a/drivers/pci/controller/dwc/pcie-dw-rockchip.c
+++ b/drivers/pci/controller/dwc/pcie-dw-rockchip.c
@@ -1566,6 +1566,8 @@ static int rk_pcie_host_config(struct rk_pcie *rk_pcie)

        /* Enable L0s capability */
        if (rk_pcie->linkcap_off) {
+               pci->n_fts[0] = 255; /* Gen1 */
+               pci->n_fts[1] = 255; /* Gen2+ */
                val = dw_pcie_readl_dbi(rk_pcie->pci, rk_pcie->linkcap_off);
                val |= PCI_EXP_LNKCAP_ASPM_L0S;
                dw_pcie_writel_dbi(rk_pcie->pci, rk_pcie->linkcap_off, val);

# 针对5.10和4.19内核使用的修复补丁如下：
--- a/drivers/pci/controller/dwc/pcie-dw-rockchip.c
+++ b/drivers/pci/controller/dwc/pcie-dw-rockchip.c
@@ -2538,6 +2538,8 @@ static int __maybe_unused rockchip_dw_pcie_resume(struct
device *dev)
 std_rc_done:
        /* Enable L0s capability */
        if (rk_pcie->linkcap_off) {
+               rk_pcie->pci->n_fts[0] = 255; /* Gen1 */
+               rk_pcie->pci->n_fts[1] = 255; /* Gen2+ */
                ret = dw_pcie_readl_dbi(rk_pcie->pci, rk_pcie->linkcap_off);
                ret |= PCI_EXP_LNKCAP_ASPM_L0S;
                dw_pcie_writel_dbi(rk_pcie->pci, rk_pcie->linkcap_off, ret);
```

2.11 禁用此设备L0s功能

```
#这里举的例子是一款wifi，其PID和VID分别为0x14d9和0x4415，实际修改请填写客诉案子所使用的PCIe
设备的PID和VID
# lspci -vvv可以获取具体设备的PID和VID
# 0002:24:00.0 Class 0c03: Device 1d49:4415 (rev 01) (prog-if 30)
--- a/drivers/pci/quirks.c
+++ b/drivers/pci/quirks.c
@@ -2399,6 +2399,13 @@ DECLARE_PCI_FIXUP_FINAL(PCI_VENDOR_ID_INTEL, 0x10f1,
quirk_disable_aspm_l0s);
 DECLARE_PCI_FIXUP_FINAL(PCI_VENDOR_ID_INTEL, 0x10f4, quirk_disable_aspm_l0s);
 DECLARE_PCI_FIXUP_FINAL(PCI_VENDOR_ID_INTEL, 0x1508, quirk_disable_aspm_l0s);

+/*
+ * CYW989459 Wireless Module based on BCM4359 802.11ac Dual-Band Wireless
+ * Network Controller doesn't work stable under L0S from test, disable L0S
+ * on the PCIe link.
+ */
+DECLARE_PCI_FIXUP_FINAL(0x14d9, 0x4415, quirk_disable_aspm_l0s);
```

2.12 检查IOMUX配置

PCIe模块的芯片信号连接，除了数据线和参考时钟差分对，可能还有以下这些低速IO：

低速IO名称	RC模式	说明
PERSTN	GPIO输出	必选，配置dts "reset-gpios"项
WAKEN	GPIO(PMU域)输入	可选，function驱动注册对应GPIO中断及唤醒源，非PCIe控制器驱动处理
PWREN	GPIO输出	可选，配置dts "vpcie3v3-supply"项
CLKREQN	FUNCTION	可选，支持L1SS时使用，配置dts "supports-clkreq"项
PRSNT	GPIO输入	可选，配置dts "prsnt-gpios"项

- 只有CLKREQN使用PCIe的function功能，该信号仅L1SS功能需要使用，否则可以不接，使用时必须添加dts对应属性；
- 其他信号都是使用*GPIO功能*。检查板级文件时，PCIe节点如果引用了这些信号的pinctrl节点，并且使用了function功能，请删除。

2.13 查看err event

若需要启用此功能，先确保包含此提交（`pcie: rockchip: dw: Add debugfs support`），否则可以在附录章节所示redmine系统中获取`0001-pcie-rockchip-dw-Add-debugfs-support.patch`。

使用方法：

1. 明确出问题的设备所处的控制器地址节点，我们仍然以RK3568 PCIe2x1控制器为例，它的APB基地址为0xfe260000。
2. 以`fe260000.pcie`为例, 进入`/sys/kernel/debug/fe260000.pcie`目录
3. `echo disable > err_event` 关闭所有的事件统计功能
4. `echo clear > err_event` 清除所有的事件统计计数

5. `echo enable > err_event` 开启所有的事件统计功能
6. 开始设备老化，复现您的异常case, 复现后请执行`cat dumpfifo`和`cat err_event`。
7. 将导出的信息与本文档附录中的Debugfs导出信息解析表进行对比，可以大致明确出现的问题。

如果复现后因为系统卡住而无法执行dumpinfo和err_event，可以使用io命令来执行：

```
###### 1. 查询控制器的DBI地址
# TRM Address Mapping 章节查询 PCIe2x1的DBI地址，有两组都可以用
| PCIe2x1_DBI | 0x3C0000000 | 4MB |
| PCIe2x1_DBI |  0xF6800000 | 4MB |

# 或者从rk3568.dtsi中获取pcie-dbi所对应的地址是0x3c0000000
pcie2x1: pcie@fe260000 {
    compatible = "rockchip,rk3568-pcie", "snps,dw-pcie";
    ...
    reg = <0x3 0xc0000000 0x0 0x400000>, <0x0 0xfe260000 0x0 0x10000>;
    reg-names = "pcie-dbi", "pcie-apb";
    ...
};

###### 2. 开始读取各个err event的统计数据
# 假设前面已经执行过clear 和 enable操作：
# echo clear  > err_event ===> io -4 0x3c00001a8 0x3 无需执行
# echo enable > err_event ===> io -4 0x3c00001a8 0x1c  && io -4 0x3c00001a8 0x3
无需执行
# 则分别利用控制器的DBI地址+0x1a8和DBI地址+0x1ac的配置，来获取统计计数

# 读取EBUF Under-run
io -4 0x3c00001a8 0x0
io -4 0x3c00001ac

# 读取EBUF Under-run
io -4 0x3c00001a8 0x0010000
io -4 0x3c00001ac

# 读取Decode Error
io -4 0x3c00001a8 0x0020000
io -4 0x3c00001ac

# 读取Running Disparity Error
io -4 0x3c00001a8 0x0030000
io -4 0x3c00001ac

# 读取SKP OS Parity Error
io -4 0x3c00001a8 0x0040000
io -4 0x3c00001ac

# 读取SYNC Header Error
io -4 0x3c00001a8 0x0050000
io -4 0x3c00001ac

# 读取CTL SKP OS Parity Error
io -4 0x3c00001a8 0x0060000
io -4 0x3c00001ac

# 读取Detect EI Infer
io -4 0x3c00001a8 0x1050000
```

```
io -4 0x3c00001ac

# 读取Receiver Error
io -4 0x3c00001a8 0x1060000
io -4 0x3c00001ac

# 读取Rx Recovery Request
io -4 0x3c00001a8 0x1070000
io -4 0x3c00001ac

# 读取N_FTS Timeout
io -4 0x3c00001a8 0x1080000
io -4 0x3c00001ac

# 读取Framing Error
io -4 0x3c00001a8 0x1090000
io -4 0x3c00001ac

# 读取Deskew Error
io -4 0x3c00001a8 0x10a0000
io -4 0x3c00001ac

# 读取BAD TLP
io -4 0x3c00001a8 0x2000000
io -4 0x3c00001ac

# 读取LCRC Error
io -4 0x3c00001a8 0x2010000
io -4 0x3c00001ac

# 读取BAD DLLP
io -4 0x3c00001a8 0x2020000
io -4 0x3c00001ac

# 读取Replay Number Rollover
io -4 0x3c00001a8 0x2030000
io -4 0x3c00001ac

# 读取Replay Timeout
io -4 0x3c00001a8 0x2040000
io -4 0x3c00001ac

# 读取Rx Nak DLLP
io -4 0x3c00001a8 0x2050000
io -4 0x3c00001ac

# 读取Tx Nak DLLP
io -4 0x3c00001a8 0x2060000
io -4 0x3c00001ac

# 读取Retry TLP
io -4 0x3c00001a8 0x2070000
io -4 0x3c00001ac

# 读取FC Timeout
io -4 0x3c00001a8 0x3000000
io -4 0x3c00001ac
```

```
# 读取Poisoned TLP
io -4 0x3c00001a8 0x3010000
io -4 0x3c00001ac

# 读取ECRC Error
io -4 0x3c00001a8 0x3020000
io -4 0x3c00001ac

# 读取Unsupported Request
io -4 0x3c00001a8 0x3030000
io -4 0x3c00001ac

# 读取Completer Abort
io -4 0x3c00001a8 0x3040000
io -4 0x3c00001ac

# 读取Completion Timeout
io -4 0x3c00001a8 0x3050000
io -4 0x3c00001ac
```

如果异常后连io命令都无法执行，建议将寄存器访问改写成代码形式，放在死机后jtag看到的访问异常代码点位进行打印

```
# 首先仍然开机后要根据之前步骤执行
echo clear  > err_event
echo enable > err_event

# 然后能够触发异常的相关代码点打上此补丁

// 定义要映射的基地址
#define BASE_ADDR_1 0x3c00001a8
#define BASE_ADDR_2 0x3c00001ac

// 定义事件码数组
static const u32 event_codes[] = {
0x0000000, 0x0010000, 0x0020000, 0x0030000, 0x0040000, 0x0050000,
0x0060000, 0x1050000, 0x1060000, 0x1070000, 0x1080000, 0x1090000,
0x10a0000, 0x2000000, 0x2010000, 0x2020000, 0x2030000, 0x2040000,
0x2050000, 0x2060000, 0x2070000, 0x3000000, 0x3010000, 0x3020000,
0x3030000, 0x3040000, 0x3050000
};

// 定义事件名称数组
static const char *event_names[] = {
    "EBUF Under-run", "EBUF Under-run", "Decode Error", "Running Disparity
Error",
    "SKP OS Parity Error", "SYNC Header Error", "CTL SKP OS Parity Error",
    "Detect EI Infer", "Receiver Error", "Rx Recovery Request", "N_FTS Timeout",
    "Framing Error", "Deskew Error", "BAD TLP", "LCRC Error", "BAD DLLP",
    "Replay Number Rollover", "Replay Timeout", "Rx Nak DLLP", "Tx Nak DLLP",
    "Retry TLP", "FC Timeout", "Poisoned TLP", "ECRC Error", "Unsupported
Request",
    "Completer Abort", "Completion Timeout"
};

//定义打印函数，如果担心来不及，可以将ioremap提前做，以及减少printk中的字符串
static int __init read_event_values_init(void)
```

```
{
    void __iomem *addr1, *addr2;
    int i;

    // 映射物理地址
    addr1 = ioremap(BASE_ADDR_1, sizeof(u32));
    if (!addr1) {
        return -ENOMEM;
    }

    addr2 = ioremap(BASE_ADDR_2, sizeof(u32));
    if (!addr2) {
        iounmap(addr1);
        return -ENOMEM;
    }

    // 循环读取每个事件的值
    for (i = 0; i < ARRAY_SIZE(event_codes); i++) {
        u32 value;

        // 写入事件码到第一个地址
        writel(event_codes[i], addr1);

        // 读取事件统计数值值
        value = readl(addr2);

        // 打印读取的数值
        printk(KERN_INFO "Reading %s: 0x%x\n", event_names[i], value);
    }

    // 取消映射
    iounmap(addr1);
    iounmap(addr2);

    return 0;
}
```

2.14 开启s2r流程打印

- 先确认引起死机的控制器是否使用的是combo PHY。一般2.0控制器都是用combo PHY，可以查询dtsi中该控制器所使用的phy节点。后缀表示复用关系，p、s、u、q分别表示PCIe、SATA、USB、QSGMII。

```
pcie2x1: pcie@fe260000 {
    ...
    phys = <&combphy2_psq PHY_TYPE_PCIE>;
    phy-names = "pcie-phy";
    ...
};
```

- 如果是使用combo PHY，则请确认携带下列提交。如果不是combo PHY，则不必核对下列提交。

```
commit 5dc8f0d04ba59e1a1fc0c74ec4a4cdafcf72919d
Author: Shawn Lin <shawn.lin@rock-chips.com>
```

```
Date:   Wed Nov 22 10:40:14 2023 +0800

    PCI: rockchip: dw: Validate phy mode in suspend
    Change-Id: Icc65c9d19ef2bcb54bfb3665d4c012f8ac0e710e

--- a/drivers/pci/controller/dwc/pcie-dw-rockchip.c
+++ b/drivers/pci/controller/dwc/pcie-dw-rockchip.c
@@ -8,6 +8,7 @@
  * Author: Simon Xue <xxm@rock-chips.com>
  */

+#include <dt-bindings/phy/phy.h>
 #include <linux/clk.h>
 #include <linux/delay.h>
 #include <linux/fs.h>
@@ -2335,6 +2336,12 @@ static int __maybe_unused rockchip_dw_pcie_suspend(struct
device *dev)
 no_l2:
        rk_pcie_disable_ltssm(rk_pcie);

+       ret = phy_validate(rk_pcie->phy, PHY_TYPE_PCIE, 0, NULL);
+       if (ret && ret != -EOPNOTSUPP) {
+               dev_err(dev, "PHY is reused by other controller, check the
dts!\n");
+               return ret;
+       }
+
        /* make sure assert phy success */
        usleep_range(200, 300);

commit f4b096ab2d5df4c04b4de123c0ded28ca72a3fe9
Author: Shawn Lin <shawn.lin@rock-chips.com>
Date:   Wed Nov 22 10:39:40 2023 +0800

    phy: rockchip: naneng-combphy: Add phy_validate support
    Change-Id: Icc2587edadcb37be0bac41c2128391f7583bc3ff

--- a/drivers/phy/rockchip/phy-rockchip-naneng-combphy.c
+++ b/drivers/phy/rockchip/phy-rockchip-naneng-combphy.c
@@ -287,9 +287,42 @@ static int rockchip_combphy_exit(struct phy *phy)
        return 0;
 }

+static const char *rockchip_combphy_mode2str(enum phy_mode mode)
+{
+       switch (mode) {
+       case PHY_TYPE_SATA:
+               return "SATA";
+       case PHY_TYPE_PCIE:
+               return "PCIe";
+       case PHY_TYPE_USB3:
+               return "USB3";
+       case PHY_TYPE_SGMII:
+       case PHY_TYPE_QSGMII:
+               return "GMII";
+       default:
+               return "Unknown";
+       }
```

```
+}
+
+static int rockchip_combphy_validate(struct phy *phy, enum phy_mode mode, int
submode,
+                                union phy_configure_opts *opts)
+{
+       struct rockchip_combphy_priv *priv = phy_get_drvdata(phy);
+
+       if (mode != priv->mode) {
+               dev_err(priv->dev, "expected mode is %s, but current mode is
%s\n",
+                       rockchip_combphy_mode2str(mode),
+                       rockchip_combphy_mode2str(priv->mode));
+               return -EINVAL;
+       }
+
+       return 0;
+}
+
 static const struct phy_ops rochchip_combphy_ops = {
        .init = rockchip_combphy_init,
        .exit = rockchip_combphy_exit,
+       .validate = rockchip_combphy_validate,
        .owner = THIS_MODULE,
 };
```

- 增加唤醒过程中访问DBI寄存器的打印

```
--- a/drivers/pci/controller/dwc/pcie-designware.h
+++ b/drivers/pci/controller/dwc/pcie-designware.h
@@ -357,10 +357,12 @@ static inline void dw_pcie_dbi_ro_wr_dis(struct dw_pcie
*pci)
        u32 reg;
        u32 val;

+       dev_err(pci->dev, "Start %s\n", __func__);
        reg = PCIE_MISC_CONTROL_1_OFF;
        val = dw_pcie_readl_dbi(pci, reg);
        val &= ~PCIE_DBI_RO_WR_EN;
        dw_pcie_writel_dbi(pci, reg, val);
+       dev_err(pci->dev, "End of %s\n", __func__);
 }

 @@ -393,10 +393,12 @@ static inline void dw_pcie_dbi_ro_wr_en(struct dw_pcie
*pci)
        u32 reg;
        u32 val;

+       dev_err(pci->dev, "Start %s\n", __func__);
        reg = PCIE_MISC_CONTROL_1_OFF;
        val = dw_pcie_readl_dbi(pci, reg);
        val |= PCIE_DBI_RO_WR_EN;
        dw_pcie_writel_dbi(pci, reg, val);
+       dev_err(pci->dev, "End of %s\n", __func__);
 }
```

- 休眠唤醒复现前，输入下列命令，开启休眠唤醒期间的控制台log输出

```
echo N > /sys/module/printk/parameters/console_suspend
```

2.15 是否访问DBI寄存器死机

休眠唤醒之后首先访问DBI寄存器的地方是rockchip_dw_pcie_resume()函数配置DBI写使能寄存器。按前述步骤，dw_pcie_dbi_ro_wr_dis()和dw_pcie_dbi_ro_wr_en()函数增加的打印，如果未见`End of dw_pcie_dbi_ro_wr_dis`或者`End of dw_pcie_dbi_ro_wr_en`输出，则表明确实是配置DBI写使能寄存器异常。

2.16 查看phy状态是否被切换

```
naneng-combphy fee20000.phy: expected mode is PCIe, but current mode is USB3
rk-pcie fe180000.pcie: PHY is reused by other controller, check the dts!
PM: dpm_run_callback(): rockchip_dw_pcie_suspend+0x0/0x410 returns -22
PM: Device fe180000.pcie failed to suspend async: error -22
PM: pm_system_irq_wakeup: 166 triggered fsc_interrupt_int_n
```

例如上述打印表明combo PHY被切换到其他IP的工作模式(USB3)，指示combo PHY的另一个复用模式USB3被开启，请检查dts配置，将USB3节点关闭。

2.17 休眠唤醒分层测试

```
# 1. 先确保内核打开了测试配置
CONFIG_PM_TEST_SUSPEND = y

# 2. 查询当前平台支持的休眠唤醒测试模式
cat /sys/power/pm_test
[none] core processors platform devices freezer
```

测试模式	功能描述	覆盖阶段	适用场景
freezer	仅模拟处理器暂停运行（CPU 冻结） 不挂起设备或执行平台级操作	- 冻结用户进程 - 暂停内核线程	验证任务调度暂停逻辑
devices	模拟处理器暂停 + 设备挂起 调用各设备的 `.suspend()` 回调函数	- 设备驱动 suspend/resume 流程 - 中断处理逻辑	验证设备驱动休眠兼容性
platform	在 devices 基础上增加平台级控制 包含 `noirq_suspend`/`noirq_resume`	- 平台固件交互 - 低功耗状态切换	测试平台相关电源管理代码
processors	扩展 platform 模式 额外模拟多核 CPU 的离线（CPU hotplug）操作	- CPU 热插拔 - SMP 核心状态管理	验证多核系统休眠稳定性
		- 系统核心组件挂	

测试模式	功能描述	覆盖阶段	适用场景
core	进入休眠前的最后一步 包含 `syscore_suspend` 系统级回调	超 （时钟/中断控制器等） - 最终硬件状态保存	调试休眠流程末端的系统级问题

```
# 3. 依次选择不同的休眠唤醒测试模式进行测试，定位问题在哪个模式下不复现
echo core > /sys/power/pm_test        #选择core
echo mem > /sys/power/state

echo platform > /sys/power/pm_test    #选择platform
echo mem > /sys/power/state

echo devices > /sys/power/pm_test     #选择devices
echo mem > /sys/power/state

echo freezer > /sys/power/pm_test     #选择freezer
echo mem > /sys/power/state
```

3. 常见因素说明

PCIe异常死机基本的原因汇总如下

3.1 Producer-Consumer模型异常

此类卡死一般发生在PCIe设备老化过程中，系统卡死后能看到CPU都停在访问PCIe外设的某寄存器上。

```
ffffff80083c2b9c:       b4000224        cbz     x4, ffffff80083c2be0
<igb_rd32+0x58>
ffffff80083c2ba0:       8b214082        add     x2, x4, w1, uxtw
ffffff80083c2ba4:       b9400042        ldr     w2, [x2]    #死机位置，访问PCIE映射的
寄存器
或者
ffffffc010a7a990:       d50331bf        dmb     oshld
ffffffc010a7a994:       92403c08        and     x8, x0, #0xffff
ffffffc010a7a998:       ca080108        eor     x8, x8, x8
ffffffc010a7a99c:       b5000008        cbnz    x8, ffffffc010a7a99c
<os_pci_read16+0x34>
ffffffc010a7a9a0:       f9400bf3        ldr     x19, [sp,#16]
ffffffc010a7a9a4:       a8c27bfd        ldp     x29, x30, [sp],#32
ffffffc010a7a9a8:       f85f8e5e        ldr     x30, [x18,#-8]!#死机位置，访问PCIe
映射的寄存器
ffffffc010a7a9ac:       d50323bf        hint    #0x1d
ffffffc010a7a9b0:       d65f03c0        ret
又或者
ffffffc01068193c:       7100069f        cmp     w20, #0x1
ffffffc010681940:       54000281        b.ne    ffffffc010681990
<pci_generic_config_read+0x8c>
ffffffc010681944:       39400108        ldrb    w8, [x8]    #死机位置，访问PCIe映射的
寄存器
```

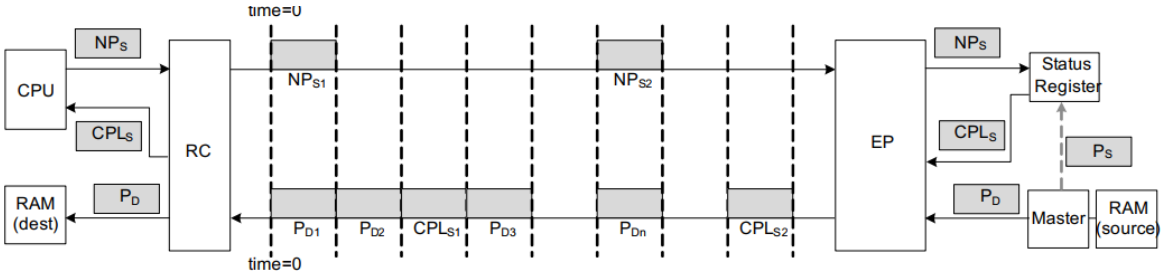
```
ffffffc010681948:       d50331bf        dmb     oshld
ffffffc01068194c:       92401d09        and     x9, x8, #0xff
ffffffc010681950:       ca090129        eor     x9, x9, x9
ffffffc010681954:       b5000009        cbnz    x9, ffffffc010681954
```

问题的产生一般是设备驱动使用了Producer-Consumer模型，且设备端存在数据发送断流的情况。下图是严格的PCIe协议上的数据包排序模型，其中有严格约束的是Row B/C/D与Col 2，尤其是D2a，即Completion包不能跨过Posted请求。

Row Pass Column?		Posted Request (Col 2)	Non-Posted Request		Completion (Col 5)
			Read Request (Col 3)	NPR with Data (Col 4)	
Posted Request (Row A)		a) No b) Y/N	Yes	Yes	a) Y/N b) Yes
Non-Posted Request	Read Request (Row B)	a) No b) Y/N	Y/N	Y/N	Y/N
	NPR with Data (Row C)	a) No b) Y/N	Y/N	Y/N	Y/N
Completion (Row D)		a) No b) Y/N	Yes	Yes	a) Y/N b) No

更通俗一点说，在Producer-Consumer模型中，作为消费者的一端，在生产者完全将数据写入某内存之前，不能使用该内存的数据；且生产者提供的状态查询寄存器也不允许在数据未完全写出去之前，提前响应消费者的状态读取请求。下图中EP设备是一个生产者，RC端是消费者：

1. EP设备从source RAM中获得数据，并采用post方式写入RC端的dest RAM，我们记为P_D。写完之后EP写一个完成标志位到它的状态寄存器(Status Register)。
2. RC端的CPU一直在轮询EP端的这个状态寄存器，来检查是否生产的数据已经写完。这个轮询的读过程我们记为NP_S。其所对应的从EP端获得的寄存器数据我们记为CPL_S。
3. 在这个流程中，CPL_S绝对不允许超过任何P_D之前发出，或者RC的CPU端在数据完全落入dest RAM之前对CPL_S进行响应。这个流程称为严格的Producer-Consumer模型。



前述问题的发生的根源，是由于设备驱动的不当业务模型所致：

- EP设备驱动和EP Firmware在使用Producer-Consumer模型时未遵守协议，当EP端向dest RAM发送P_D间隙，RC端的设备驱动恰好发出了NP_S读完成标志的请求(包括CfgRd或MemRd类型)，而EP端在P_D未完全发完之前提前允许返回了CPL_S。
- 此时，RC端因为严格遵守Producer-Consumer协议模型，等不到完整的P_D而无法接收CPL_S，且没有超时返回的途径，导致等待CPL_S的CPU卡住。
- 还需要特别注意，EP设备驱动如果不是发起读取完成标志的TLP请求，即不希望此包进入严格Producer-Consumer模型排序的包，都应该配置成Relaxed Ordering, 即TLP的header的Attr置上RO

位。否则不仅可能会因为排序而导致业务性能下降，也会导致误进入排序规则而引起不必要的异常风险。

因此，为了要打破这种协议约束下的严格模型，我们RC端提供了可修改排序规则的寄存器，允许在Producer-Consumer模型下使得CPL_S 超越 P_D ，让CPU端提前获得Status Register的数据，用于确认问题和提供临时规避方案。但需要指出，使用我们下方提供的规避方案，在Producer-Consumer模型下虽然不会因为数据推送间隙中提前返回CPL_S而卡住RC端CPU，但可能会发生数据完整性异常。例如明显CPL_S 抢先返回了完成标志位，然而P_{Dn} 却还没有完全落入dest RAM的情况，那么RC端CPU去获取dest RAM内的数据就**可能**是不完整的，缺失P_{Dn}部分的数据。这个可能性大小受CPU运算速度，驱动软件获得完成标志后去取数据流的软件延时，芯片内Memory latency/QoS等多种因素影响。目前已知一些网络设备存在这类问题，规避方案配合低概率的数据错误率、网络协议栈数据包的校验和出错处理机制，并不会最终导致使用异常。但是其他设备并不能保证，这依赖于这些设备驱动完善的校验和恢复机制。因此我们对于使用这种协议模型又无法满足约束的EP设备的建议：

- 设备驱动可以抛弃传统的Producer-Consumer模型，即以读取寄存器来确认数据发送完成的方法，改用EP端触发MSI(-x)中断机制进行通知，这是目前绝大多数PCIe设备的处理共识。
- 若确实因为技术支持层面无法修改的，请与Vendor确认其非严格约束的Producer-Consumer模式采用我们的规避方案之后，需评估驱动对数据完整性出错的处理机制是否完善，是否会严重影响业务生态的使用。

3.2 L0s的支持异常

L0s分为TX和RX两个方向。

- 当双方进入L0s, RC希望退出EIOS进行通信，则RC会让它的TX先退出L0s，退出方式是通过TX向EP的RX发送EP所需的FTS数量，则EP的RX退出L0s。
- RC的TX发出访问请求，EP从RX接受请求
- EP需要把数据返回给RC，则EP需要通过EP的TX，让RC的RX退出L0s, 则EP通过其TX向RC的RX发送RC所需个数的FTS包
- RC的RC退出L0s，并接受来自EP的TX端返回的数据。

这个流程中一旦双方有任何一端无法退出，则会导致链路在L0s和recovery之间跳转，导致PCIe传输pending住而无法超时退出。目前常规情况下我们的SDK将主控的FTS增到最大，理论上应该可以满足。如果主控已经增加到最大依然无法满足，只能禁用掉该设备的L0s。

3.3 总线阻塞

可能有其他IP由于软硬件设计因素，在物理传输未真正结束时候关闭了其时钟，导致总线被hold住，影响到了PCIe模块(可能还包括其他模块)，此时传输pending应该能够看到异常。例如ABC模块关闭时钟前需要检查自身寄存器，是否完成了传输；如果自身无法完成查询，可以查询NOC寄存器中自身的pending状态。NOC寄存器pending位一般位于GRF(General Register Files)章节，检索关键词`pending transactions`或者`nopending`, 找到对应目标模块传输pending所在寄存器和偏移。

```
+ static void __iomem *read_addr_noc; //Noc中查询该模块pending位的寄存器
/* 关时钟可能是传输后主动调用，也可能是挂在auto runtime PM里面进行管理 */
static int ABC_clk_off(struct mpp_dev *mpp)
{
        ...
+       while(i--) {
+           noc1 = readl(read_addr_noc + 0x4); //举例子 read_addr_noc =
ioremap(0x2600a074, 0x4)
```

```
+           if ((noc1 & 0x8) == 0) // 该ABC模块pending位是bit 3, 0表示有传输被阻塞
+               pr_err("ABC's transfer is pending\n");
+           else
+               break;
+       }
+       pr_err("%s %d noc1=0x%x\n", __func__, __LINE__, noc1);
+       clk_disable(clk);

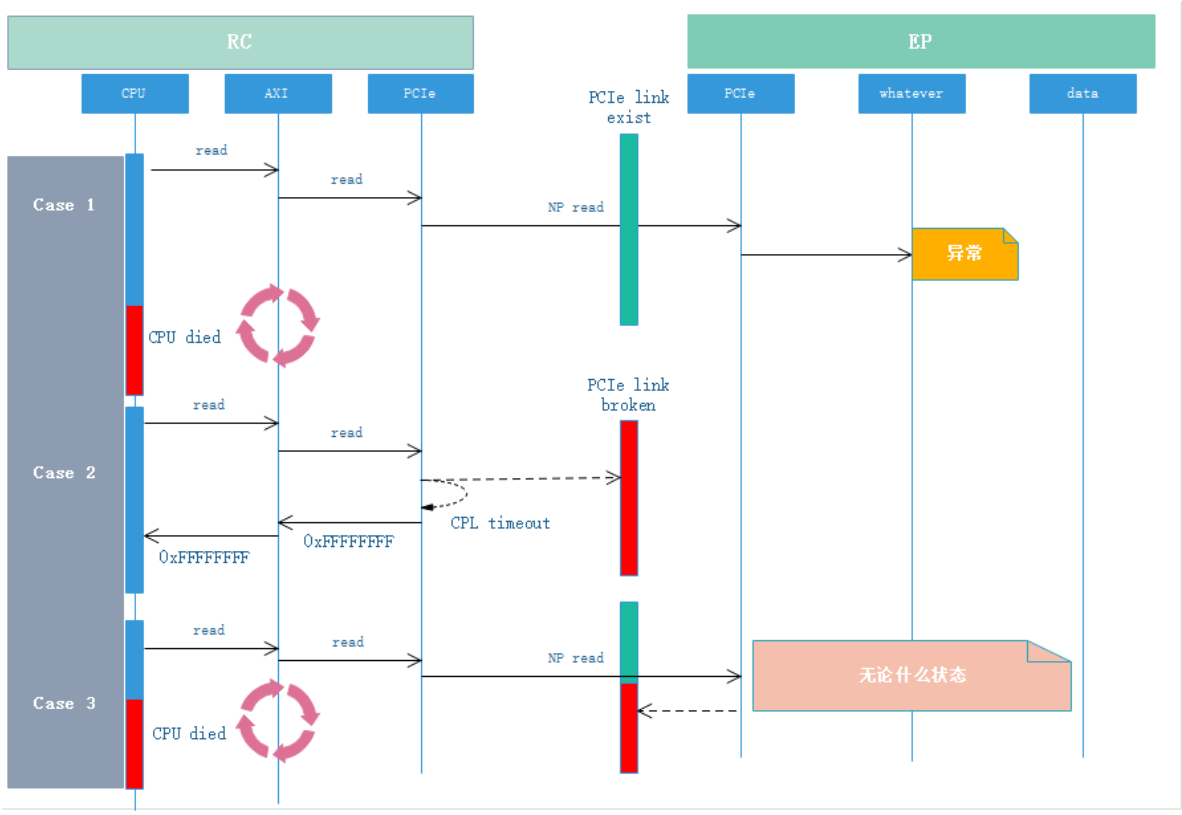
        ...
}
```

3.4 链路或者外设异常

链路或者外设某个时间点异常也会导致主控发出去的读访问无法超时退出。此时CPU应该停在访问PCIe外设的BAR地址空间上，典型的堆栈如下：

```
Thread #1 (Suspended::Breakpoint)
-----------------------------------
__raw_readl()             at io.h:75          0xffffffffc009559b2c
readl()                   at io.h:223         0xffffffffc009559b2c
ioread32()                at io.h:811         0xffffffffc009559be4
ata_wait_register()       at libata-core.c:6667  0xffffffffc009559be4
ahci_stop_engine()        at libahci.c:731    0xffffffffc008bb3e28
ahci_deinit_port()        at libahci.c:954    0xffffffffc008bb2650
ahci_port_stop()          at libahci.c:2488   0xffffffffc008bb2734
ata_pci_shutdown_one()    at libata-core.c:6188 0xffffffffc008b9258
ahci_shutdown_one()       at ahci.c:1988      0xffffffffc008bafc4
pci_device_shutdown()     at pci-driver.c:520   0xffffffffc0087d74a8
device_shutdown()         at core.c:4831      0xffffffffc008aee118
kernel_restart_prepare()  at reboot.c:87      0xffffffffc008079fa8
kernel_restart()          at reboot.c:268     0xffffffffc008079fa8
__do_sys_reboot()         at reboot.c:738     0xffffffffc00807a2f0
__se_sys_reboot()         at reboot.c:701     0xffffffffc00807a558
__arm64_sys_reboot()      at reboot.c:701     0xffffffffc00807a558
invoke_syscall()          at syscall.c:38     0xffffffffc008023e7c
invoke_syscall()          at syscall.c:52     0xffffffffc008023e7c
el0_svc_common()          at syscall.c:142    0xffffffffc008023f8c
do_el0_svc()              at syscall.c:206    0xffffffffc0080240e4
el0_svc()                 at entry-common.c:637 0xffffffffc00957d3f4
el0t_64_sync_handler()    at entry-common.c:655 0xffffffffc00957e570
el0t_64_sync()            at entry.S:585      0xffffffffc0080115a4
el0t_64_sync()            at entry.S:585      0xffffffffc0080115a4
```

- case1: 链路处于正常可通信状态(一般指L0或者L1)，则EP端内部异常不会导致主控发出的写访问被hold住，但是主控发出的读访问会被hold住，无法超时退出。
- case2: 主控发出访问之前，如果链路已经断开，则不会导致主控发出的访问被hold住。
- case3: 如果主控发出访问时链路正常，但是EP接收访问请求后链路异常，则主控的访问会被hold住，无法超时退出。



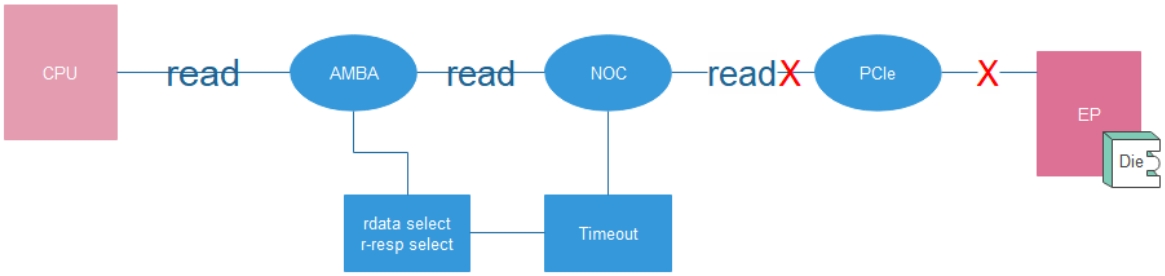
真正能够导致主控异常的只有case1和3，case 1触发的概率与外设及其驱动设计强相关，case 3的概率则较低。处理此问题需要从三个方面着手，其一是外设工作稳定、板载信号稳定；其二是规避外设不允许使用的异常流程；其三是芯片有超时机制能应对此类问题(兜底方案)。但是需要特别说明，超时时间需要覆盖各类场景，包括低功耗退出、外设回复主控访问的最长时延, 否则会出现：

- 系统通过AXI总线向RC发出访问EP设备的请求
- EP退出低功耗--->取数据时间超过超时时间
- RC产生超时，释放AXI总线请求，系统访问提前结束
- EP返回数据到RC，RC向AXI总线发送EP返回的数据，AXI因无主请求而宕机

芯片	是否支持超时	超时时间	使能寄存器	比特(写1使能，高16位有写掩码)	超时后返回的数据	备注
RK1808	x	x	x	x	x	
RK3528	x	x	x	x	x	
RK356X	x	x	x	x	x	

芯片	是否支持超时	超时时间	使能寄存器	比特(写1使能，高16位有写掩码)	超时后返回的数据	备注
RK3572	√	可配	0x26010028	bit9: pcie1 dbi 超时 bit8: pcie1 slave 超时 bit7: pcie0 dbi 超时 bit6: pcie0 slave 超时	可选全0或者全1	- 默认超时时间1.4s，最大可配超时时间44.7s - 软件默认开启了超时支持 - 超时后返回全0或者全1的选择寄存器(rdata select)是0x26018004的bit0：1表示返回全1，0表示返回全0，高16位有写掩码 - 超时后可选择返回给CPU的错误码error code的寄存器(r-resp select)是0x26018004的bit1：0表示返回error code给CPU，1表示返回normal response给CPU，高16位有写掩码 - 依据上述配置选择，当超时发生后，如果CPU收到error code会打印调用栈方便debug；如果CPU收到normal response, 则CPU可以收到返回的数据，程序需自行判断返回值是否全0或者全1，进行容错处理
RK3576	√	166us 不可配	0x2600A030	bit3: pcie1 dbi 超时 bit4: pcie1 slave 超时 bit2: pcie0 dbi 超时 bit1: pcie0 slave 超时	全0	- 超时太短导致使用场景受限(例如Wi-Fi L1SS退出时间太久) - 超时返回的数据全0，不符合PCIe协议约束的链路异常行为，需要逐个外设驱动适配修改出错处理流程 - CPU收到error code, 触发异常，要整体屏蔽
RK3588	x	x	x	x	x	

针对例如RK3572这样开始支持超时控制的平台，其结构说明如下：



默认系统软件默认启用Timeout模块，并将rdata select配成1，即返回全1数据给CPU；而r-resp select默认配置成0，即返回error code给CPU，会打印出调用栈后卡住，用于debug问题。如果客户想直接进行软件workaround, 需要将r-resp select配置成1，此时CPU可以收到normal response, 不会卡住，且得到全1的rdata，可以以此来判断发生了异常，进而进行异常容错处理和恢复。具体配置的寄存器请查阅上表中各芯片栏的备注部分。

3.5 PHY的参考时钟异常

PCIe控制器的PHY的100MHz参考时钟异常会导致控制器异常，CPU访问外设BAR地址或者控制器DBI地址卡住。PCIe 2.0 PHY的参考时钟是芯片内部PLL提供，而3.0 PHY的参考时钟一般是外部时钟芯片提供。需要特别注意例如RK3588的复杂拆分模式下，3.0 PHY的参考时钟属于多个控制器的共同资源，其使能脚的控制需要参照《Rockchip_Developer_Guide_PCIe_CN.pdf》中DTS配置范例，并仔细阅读注意事项。

3.6 功能引脚被误激活

如果在PCIe控制器用于RC模式时，PERSTN引脚和WAKEN引脚被配置为功能脚，有可能让外设错误地对系统总线进行复位，又或者WAKEN引脚被外设错误拉低，导致控制器在被关闭(复位或者无时钟)状态下被激活，均可能导致系统总线挂死。尤其是在不接外设时，这些处于功能状态的引脚是悬空态，板子温度变化和其他外设的信号干扰均可能使得其输入状态不稳定，误触发其功能，导致系统总线挂死。

4. 解决方法

1. 按照上述排查项目和常见因素说明，已明晰发生异常原因的，按需修复软硬件缺陷。
2. 如果确实存在极低概率异常，无法查出异常原因，且发生了排查项目中所述synchronous external abort的平台，可以关闭PCIe模块产生synchronous external abort，并让EP原厂帮忙在EP驱动中调用相关API进行设备复位。

```
/* 移除设备 */
pci_lock_rescan_remove();
pci_stop_and_remove_bus_device();
pci_unlock_rescan_remove();
/* 保存EP设备的配置空间 */
pci_save_state()
/* 恢复EP设备的配置空间 */
pci_restore_state()
/* 复位设备 */
rockchip_dw_pcie_pm_ctrl_for_user()
```

5. 附录

5.1 开发资源获取地址

取得redmine权限后可以直接访问https://redmine.rock-chips.com/documents/107获取前文所述各类资料。

5.2 LTSSM状态机

状态描述符	状态代码 (6'h)	备注
S_DETECT_QUIET	0x00	未检到外设RX端接电阻，设备未正常工作
S_DETECT_ACTIVE	0x01	-
S_POLL_ACTIVE	0x02	-
S_POLL_COMPLIANCE	0x03	兼容性测试模式。非测试模式下因信号异常原因错误进入
S_POLL_CONFIG	0x04	-
S_PRE_DETECT_QUIET	0x05	-
S_DETECT_WAIT	0x06	-
S_CFG_LINKWD_START	0x07	-
S_CFG_LINKWD_ACEPT	0x08	Lane 顺序检测过程
S_CFG_LANENUM_WAIT	0x09	-
S_CFG_LANENUM_ACEPT	0x0a	-
S_CFG_COMPLETE	0x0b	物理层检测完毕
S_CFG_IDLE	0x0c	-
S_RCVRY_LOCK	0x0d	速率切换过程
S_RCVRY_SPEED	0x0e	-
S_RCVRY_RCVRCFG	0x0f	-
S_RCVRY_IDLE	0x10	-
S_RCVRY_EQ0	0x20	-
S_RCVRY_EQ1	0x21	-
S_RCVRY_EQ2	0x22	-
S_RCVRY_EQ3	0x23	-
S_L0	0x11	链路正常工作过状态L0
S_L0S	0x12	-
S_L123_SEND_EIDLE	0x13	-
S_L1_IDLE	0x14	-
S_L2_IDLE	0x15	-
S_L2_WAKE	0x16	-
S_DISABLED_ENTRY	0x17	-
S_DISABLED_IDLE	0x18	-

状态描述符	状态代码 (6'h)	备注
S_DISABLED	0x19	-
S_LPBK_ENTRY	0x1a	-
S_LPBK_ACTIVE	0x1b	loopback测试模式，一般在测试环境下会进入
S_LPBK_EXIT	0x1c	-
S_LPBK_EXIT_TIMEOUT	0x1d	-
S_HOT_RESET_ENTRY	0x1e	外设主动发其hot reset流程
S_HOT_RESET	0x1f	-

5.3 Debugfs导出信息解析表

事件符号	含义
EBUF Overflow	-
EBUF Under-run	-
Decode Error	包解码错误，信号异常
Running Disparity Error	极性偏差错误，8bit/10bit编码中0与1数量比例失衡，Gen2信号异常
SKP OS Parity Error	SKP 序列奇偶校验错误，Gen3信号异常
SYNC Header Error	异步包头错误，信号问题
CTL SKP OS Parity Error	控制的SKP 序列奇偶校验错误，Gen3信号异常
Detect EI Infer	检测到信号串扰
Receiver Error	接收端错误
Rx Recovery Request	RX信号收到外设请求，进入rescoery状态进行信号矫正
N_FTS Timeout	外设的n_fts不达标，L0s回到L0异常进入recovery状态
Framing Error	帧格式解码错误，信号异常
Deskew Error	deskew错误，信号异常
BAD TLP	收到错误TLP包
LCRC Error	link CRC，信号异常
BAD DLLP	错误的DLLP包，信号异常
Replay Number Rollover	重传包积累太多，replay buffer冲了
Replay Timeout	误码包重传超时。
Rx Nak DLLP	收到来自下游的DLLP而拒绝其访问
Tx Nak DLLP	发往下游的DLLP被拒绝
Retry TLP	重传的TLP包数量，可以表征误码率
FC Timeout	更新流控超时
Poisoned TLP	上报一个错误的TLP类型，可能是因为链路原因或RAM的位跳变引起
ECRC Error	end CRC，信号异常
Unsupported Request	收到不支持的请求类型的TLP包，可能是访问外设被拒绝
Completer Abort	设备接收到了RC发来的请求，但是发生了其内部错误，要求终止该访问请求。
Completion Timeout	读取外设相关资源，TLP返回超时
Common event signal status	读取目前控制器的PM模式